OPIPE-99: Best Practice Summary

> **Series:** OPIPE — OpenPipeline Beyond Logs | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 10/26/2026

Definitive best practice settings for OpenPipeline beyond logs — spans, metrics, events, and cross-scope patterns. Each entry specifies the exact configuration.

Table of Contents

1. Multi-Scope Architecture
2. Matching Conditions
3. Span Filtering & Enrichment
4. Sampling-Aware Metrics & RED
5. Cardinality Management
6. Security Context
7. Business & Security Events
8. Cross-Scope Patterns
9. Ingestion vs Query-Time
10. Production Readiness

---

1. Multi-Scope Architecture

Practice	Recommended Setting/Value	Priority
One pipeline per source type per scope	Separate pipelines for each distinct data source	Critical

Practice	Recommended Setting/Value	Priority
Never leave >20% of volume on the default route	Target 0% in the default bucket for production environments	Critical
Route ordering: most specific first	Security/audit → application → infrastructure → API-ingested → default catch-all	Critical
Configure each scope independently	Logs, Spans, Metrics, Events, Business Events, Security Events, and the other configuration scopes — changes in one scope have zero effect on others	Recommended
Know the fixed stage order	Ingest → Routing → pipeline → Storage. Inside the pipeline the stage sequence is fixed: Processing (mask, drop, transform) → Smartscape node → Smartscape edge → Permission → Product allocation → Cost allocation → Bucket assignment → Metric extraction → Davis → Data extraction	Critical
Know the default bucket retention	Built-in buckets; on the validation tenant `default_logs` keeps 35 days and `default_spans` 10 days — read yours with `fetch dt.system.buckets`	Recommended
Audit default bucket percentage weekly	`countIf(dt.system.bucket == "default_logs")/count()` — CRITICAL if >80%, WARNING if >50%	Critical
Check pipeline count per scope	`countDistinct(dt.openpipeline.pipelines)` — as a community rule of thumb, 3+ for logs and 2+ for spans	Recommended

2. Matching Conditions

Practice	Recommended Setting/Value	Priority
Scope each processor with its own matching condition	There is no processor-group construct — every processor carries a DQL matcher; `true` applies it to all records	Critical

Practice	Recommended Setting/Value	Priority
Parse once, match on the result	Extract the distinguishing value into a field early in Processing (`dt.temp.*` if it should not be stored, SaaS 1.345+) and match later processors on it	Recommended
Know which stages are first-match	Permission, Product allocation, Cost allocation and Bucket assignment run only the first matching processor — list specific conditions first	Critical
Use conditional bucket assignment when only the destination differs	Conditional **Bucket assignment** processors (first match wins) inside one pipeline, instead of a pipeline per bucket	Recommended
Make matchers in all-matches stages mutually exclusive	Unless multi-match is intentional — overlapping processors can set conflicting field values	Recommended
Use separate pipelines when lifecycle or owner differs	Different bucket, retention, security context or owning team = separate pipeline	Critical
Use pipeline groups for centrally mandated stages	A composition of base pipelines can restrict or mandate stages for member pipelines	Recommended
Exclude every mandated stage from members	A stage enabled in both a base and a member pipeline runs in both — exclude it in the group's `memberStages` (prefer an `include` list) — see OPIPE-07	Critical
Keep Permission, cost, product and bucket central	Run them in base pipelines; they come before every extraction stage, so member-extracted data sees the central values	Critical
Route only to member pipelines	Base pipelines are not routable; first matching route wins; monitor the share of records on the default route	Critical
Platform team owns the routing table	Routing has no owner-based access control — write access to a scope's routing covers every route	Critical

Practice	Recommended Setting/Value	Priority
Decide the role at creation	`groupRole` and `routing` are permanent; a member belongs to one group, a base pipeline can serve several	Recommended
Budget the limits	100 processors per base pipeline, 10 slots per group, 100 pipelines per scope, extraction in at most 5 pipelines per record	Recommended
Gate member pipelines in CI	Metric-key prefix, Davis allow-list, dimension deny-list, processor counts — warn first, then fail	Recommended

3. Span Filtering & Enrichment

Practice	Recommended Setting/Value	Priority
Drop health check spans	Drop: `url.path` matches `"*health*"` OR `"/ready*"` OR `"/livez*"`, or `span.name` matches `"grpc.health.v1.Health/*"` — HTTP span names often carry only the method, so match on the path	Critical
Drop metrics scraping spans	Drop: `url.path` matches `"/metrics*"` OR `"/actuator/*"`	Critical
Drop OPTIONS preflight spans	Drop: `http.request.method == "OPTIONS"`	Recommended
Drop synthetic/heartbeat spans	Drop: `span.name` matches `"*synthetic*"` or `"*heartbeat*"`	Recommended
Quantify noise before configuring drops	Count health checks, scrapes, OPTIONS as % of total — measure your own share	Recommended
Route spans to purpose buckets	Frontend: `frontend_spans` 14d. API: `api_spans` 35d. DB: `db_spans` 14d. Default: built-in retention	Recommended

Practice	Recommended Setting/Value	Priority
Add `business.domain` enrichment	`matchesValue(service.name, "checkout*")` → `business.domain = "commerce"` (`service.name` is on OTel spans only — use `dt.service.name` where that is what your spans carry)	Recommended
Add `environment` enrichment	`matchesValue(k8s.namespace.name, "*prod*")` → `environment = "production"`	Recommended
Pre-classify severity	`http.response.status_code >= 500` → `incident.severity = "high"`	Optional
Prefer metric extraction over event generation	Extraction: 1 data point/interval (low cost). Generation: 1 record/span (high cost)	Critical

4. Sampling-Aware Metrics & RED

Practice	Recommended Setting/Value	Priority
Enable sampling-aware for count metrics	`span.request_count` , `span.error_count` must weight each span by 1 / its sampling probability (ATM stores the ratio as a denominator: 16 = 1 in 16) — enable both **Account for aggregated spans** and **Extrapolate metrics to account for trace sampling** on the counter processor	Critical
Use the sampling-aware histogram for duration	Set **Measurement: Duration**, which enables the sampling options automatically. Percentiles are representative either way; extrapolated counts and sums are not without it	Recommended
Check spans for sampling metadata	`dt.system.sampling_ratio` is the query's read sampling, not the trace sampler — check `supportability.atm_sampling_ratio` and `sampling.threshold`	Critical

Practice	Recommended Setting/Value	Priority
Check OTel spans for sampling metadata	OpenTelemetry spans usually carry no sampling-rate metadata, so extrapolation is less effective on them	Recommended
Error rate ratios tolerate uniform sampling only	Accurate when every request had the same chance of being kept; biased under tail sampling or when ATM samples request types at different rates	Recommended
Rate (RED)	Metric: `span.request_count`, Sampling aware counter, `span.kind == "server"` AND `isNotNull(endpoint.name)` (request roots — one request can produce several server spans), dims: `dt.service.name`, `http.route`, both sampling options enabled	Critical
Errors (RED)	Metric: `span.error_count`, Sampling aware counter, `span.kind == "server"` AND `isNotNull(endpoint.name)` AND `status_code >= 500`, dims: `dt.service.name`, `http.route`, `status_code`, both sampling options enabled	Critical
Duration (RED)	Metric: `span.duration`, Sampling aware histogram (Measurement: Duration), `span.kind == "server"` AND `isNotNull(endpoint.name)`, dims: `dt.service.name`, `http.route`, sampling-aware: Yes	Critical
Validate against built-in metrics	Compare `span.request_count` vs `dt.service.request.count` — significant divergence = misconfiguration. Service metrics are themselves *"based on captured traces"*, so low-frequency requests can differ in both	Critical

5. Cardinality Management

Practice	Recommended Setting/Value	Priority
Never use `trace.id`, `span.id`, or `content` as dimensions	Unique per record = unbounded cardinality — past the 1M per-metric limit on Metrics Classic, and what Grail *"may restrict or reject"*	Critical
Never use `http.url` as dimension	Use `http.route` (pattern) instead of `http.url` (instance with query params)	Critical
Never use `k8s.pod.name` as a metric dimension	Use `k8s.deployment.name` (or `k8s.workload.name`); do not rewrite `k8s.pod.name` itself	Critical
Remove `http.url` when `http.route` exists	**Remove fields** processor in the Processing stage	Recommended
Remove captured header attributes from spans	List each `http.request.header.<name>` attribute by name — high-cardinality, rarely needed post-ingestion	Recommended
Normalize URL paths to patterns	`/users/12345/orders` → `/users/{id}/orders` in a DQL processor	Critical
Group status codes into classes	`200/201/204` → `2xx`, `400/401/403` → `4xx`, `500/502/503` → `5xx`	Recommended
Cardinality budget: SLO metrics (community practice)	<1,000 series, max 2-3 dimensions	Recommended
Cardinality budget: operational metrics (community practice)	<10,000 series, max 3-4 dimensions	Recommended
Cardinality budget: exploratory metrics (community practice)	<50,000 series, max 4-5 dimensions	Recommended
Run cardinality estimation before extraction	`countDistinct()` across proposed dimensions on 1h data — reject if >100,000	Critical

Practice	Recommended Setting/Value	Priority
Bucket continuous values into tiers	Duration: <100ms=fast, 100-500ms=normal, 500ms-2s=slow, >2s=very_slow	Recommended
Hashing pseudonymizes PII (salt it) but does NOT reduce cardinality	An unsalted hash of an IP or email can be reversed by enumeration — use a secret salt or drop the field. 100K emails → 100K hashes: use removal/grouping to reduce	Recommended

6. Security Context

Practice	Recommended Setting/Value	Priority
Set `dt.security_context` on every scope with sensitive data	The **Permission** stage in Logs, Spans, Metrics, Events, Bizevents — or a `dt.security_context` host tag, which applies to all logs, spans, metrics and events from that host	Critical
Extension metrics: set explicitly	Data does not inherit an entity's security context. Set `dt.security_context` per extension configuration, or in the Metrics scope Permission stage on a `metric.key` prefix	Critical
Use MATCH operator for array values	`=`, `STARTSWITH`, `IN` always return false for array-valued security contexts	Critical
Business events: by event type	Payment events → `"finance-team"`, signup → `"marketing-team"`, default → `"product-team"`	Recommended
Spans: by namespace or service	`k8s.namespace.name` matches `"team-alpha-*"` → `"team-alpha"`	Recommended

7. Business & Security Events

Practice	Recommended Setting/Value	Priority
Enrich business events with channel	`event.provider == "mobile-app"` → `channel = "mobile"`	Recommended
Extract conversion funnel metrics	Separate count per stage: views → adds → checkouts → purchases	Recommended
Extract revenue with a Value metric	`bizevent.purchase_revenue` on `order.total`, dims: `provider`, `channel`; sum at query time	Recommended
Business event retention	90-365 days depending on reporting needs	Recommended
Never drop security events	No drop rules on any security pipeline	Critical
Never sample security events	Every individual event is potentially significant	Critical
Security event retention	Vulnerability and detection findings: 365 days. Compliance: 2555 days (7 years). Platform `AUDIT_EVENT`s are in `dt.system.events`, not here	Critical
Align to compliance frameworks	Confirm with your compliance team. Commonly cited: PCI-DSS one year (three months immediately available); SOC 2 no fixed period; HIPAA six years for required documentation; GDPR no longer than necessary	Critical
Security context on all security pipelines	`dt.security_context = "security-team"`	Critical

8. Cross-Scope Patterns

Practice	Recommended Setting/Value	Priority
Primary correlation key: `dt.smartscape.service`	`dt.entity.service` is `deprecated`. Present on every span but only on logs from processes mapped to a service — measure coverage per scope before relying on it	Critical
Give logs a service key if missing	Set it at the source (OneAgent primary tags) or derive it in a DQL processor — OpenPipeline does not look up entities at ingestion	Recommended
Correlate spans with workloads	Spans carry no `k8s.deployment.name` — join on `k8s.namespace.name` + `k8s.pod.name`, or map with an Inline lookup processor	Recommended
Extract same metric from two scopes for validation	`log.error_count` from logs + `span.error_count` from spans — divergence = pipeline issue	Recommended
Name related buckets with common prefix	`checkout_logs`, `checkout_spans`, `checkout_events` — enables `storage:bucket-name startsWith "checkout_"` IAM conditions	Recommended
Expire related buckets in logical sequence	Spans first (14d), logs (35d), events (35d), business events (90d)	Recommended
Use DQL for cross-scope correlation	OpenPipeline cannot join across scopes — use DQL `lookup` and `append`	Critical

9. Ingestion vs Query-Time

Practice	Recommended Setting/Value	Priority
Mask at ingestion — no exceptions	PII must be redacted before storage	Critical

Practice	Recommended Setting/Value	Priority
Drop noise at ingestion	Debug, health checks, synthetic — never store and filter later	Critical
Set security context at ingestion only	Cannot be added retroactively to stored data	Critical
Extract metrics at ingestion only	Cannot create metrics from historical data	Critical
Route to buckets at ingestion only	Bucket assignment is permanent — cannot move data after storage	Critical
Keep raw for exploratory analysis	If you don't know what you're looking for, store raw and DQL at query time	Recommended
Decision order	(1) Sensitive? Mask. (2) Noise? Drop. (3) Access control? Security context. (4) Need metric? Extract. (5) Everything else? Store raw.	Critical

10. Production Readiness

Practice	Recommended Setting/Value	Priority
No overlapping routing conditions	Audit conditions for overlap — first match wins, order-dependent behavior is a bug	Critical
Drop rules before extraction rules	Filter noise before creating metrics — otherwise you extract from noise	Critical
Volume monitoring on every pipeline	`makeTimeseries count(), by: {dt.openpipeline.pipelines}, interval:1h` — alert on spikes/drops	Critical

Practice	Recommended Setting/Value	Priority
Monitor what was not stored	Self-monitoring metrics `dt.sfm.openpipeline.pipelines_out.records` (by `pipeline_id`, `bucket_name`) and `dt.sfm.openpipeline.not_stored.records` (by `configuration`, `reason`) — stored-record counts cannot see dropped records	Recommended
Document rollback plan before changes	Record previous config — OpenPipeline cannot reprocess historical data	Critical
Metric naming convention	`scope.metric_name` (e.g., `span.request_count`, `log.error_count`)	Recommended
Test cascade chains end-to-end	Span → event → metric → SLO — failures propagate silently	Critical

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.